

Объяснение реализации PCA

NumLA, Лабораторная работа №1

1 Общая идея

PCA сводится к цепочке шагов: **центрировать данные → (опционально) отмасштабировать → найти собственные векторы матрицы ковариаций → спроецировать / восстановить данные**. Модуль `pca.py` реализует эту цепочку в виде четырёх функций (`pca_fit`, `pca_transform`, `pca_inverse`, `explained_variance_ratio`) и двух вспомогательных (`_mgs_orthonormalize`, `_fix_signs`). Прикладные скрипты (`morphometry.py`, `eigenfaces.py`, `manifold.py`) применяют это ядро к трём наборам данных.

2 Часть 1. pса.py — ядро

2.1 Константа TAU_REL

```
TAU_REL = 1e-12
```

Множитель для порога численного нуля:

$$\tau_\lambda = 10^{-12} \max_i |\lambda_i|$$

Собственные значения, по модулю меньше этого порога, считаются нулевыми (шумом округления), а не настоящими направлениями с ненулевой дисперсией — ровно то соглашение, которое задано в методичке.

2.2 _mgs_orthonormalize(V) — ортонормирование Грамом-Шмидтом

```
def _mgs_orthonormalize(V):
    m, q = V.shape
    Q = np.zeros_like(V)
    for j in range(q):
        v = V[:, j].copy()
        for _ in range(2):
            for i in range(j):
                v = v - (Q[:, i] @ v) * Q[:, i]
            Q[:, j] = v / np.linalg.norm(v)
    return Q
```

Это **модифицированный метод Грама-Шмидта (MGS)**, применённый дважды («reorthogonalization»), как явно требует задание в пункте «Приём с матрицей Грама»:

«ортogonalизуйте каждый полученный вектор относительно предыдущих двумя проходами модифицированного метода Грама-Шмидта, затем нормируйте»

Зачем это нужно. При приёме Грама каждый вектор

$$u = \frac{X_s^\top w}{\sqrt{(n-1)\lambda}}$$

вычисляется **независимо** от остальных через отдельную формулу, а не как часть единого ортогонального разложения (в отличие от прямого `eigh(Σ)`, который сразу даёт ортонормированный базис). Из-за ошибок округления, особенно при близких λ_i , такие векторы могут оказаться не строго ортогональны друг другу. Один проход Грама-Шмидта в этой ситуации может быть численно неустойчив (классическая проблема потери ортогональности), поэтому применяется **два** прохода — стандартный приём для существенного повышения точности.

Как это работает построчно. Для каждого следующего столбца j :

1. Берём исходный вектор $v = V[:, j]$.
2. Два раза подряд вычитаем из него проекции на уже построенные ортонормированные столбцы $Q[:, :j]$: $v -= (Q[:, i] @ v) * Q[:, i]$.
3. Нормируем результат на единичную длину.

2.3 `_fix_signs(U)` — фиксация знака

```
def _fix_signs(U):  
    U = U.copy()  
    for j in range(U.shape[1]):  
        idx = np.argmax(np.abs(U[:, j]))  
        if U[idx, j] < 0:  
            U[:, j] = -U[:, j]  
    return U
```

Собственный вектор определён с точностью до знака: u и $-u$ задают одно и то же направление (см. раздел методички «Что делает результат неоднозначным»). Чтобы результат был воспроизводимым, задано соглашение:

«координата с наибольшим модулем должна быть положительна. При равенстве модулей выбирается координата с наименьшим индексом»

`np.argmax` при наличии нескольких равных максимумов всегда возвращает **первый** (наименьший) индекс — это ровно требуемое поведение, дополнительной обработки равенства модулей не нужно.

2.4 pca_fit(X, k, standardize, Gram_trick) — основная функция

2.4.1 Проверка формы входа

```
X = np.asarray(X, dtype=float)

if X.ndim != 2:
    raise ValueError(f"X must be a 2D array (n, m), got array with ndim={X.ndim}.")

n, m = X.shape

if n < 2:
    raise ValueError(f"Invalid sample count: n={n}, but at least n=2 observations "
                    f"are required.")

if m < 1:
    raise ValueError(f"Invalid feature count: m={m}, but at least m=1 feature "
                    f"is required.")
```

Задание требует $n \geq 2$. Изначально это было реализовано голым `assert n >= 2` — но `assert` без сообщения (а тем более отключаемый флагом `python -O`) не «объясняет», что пошло не так. Автотесты (`test_invalid_sample_or_feature_count`) явно проверяют, что на некорректный вход (недопустимое число наблюдений **или** признаков) кидается понятная ошибка — поэтому обе проверки заменены на `raise ValueError` с текстом, плюс добавлена симметричная проверка $m \geq 1$, которой раньше не было вовсе (без неё пустой по признакам массив упал бы позже с невнятной ошибкой вроде `IndexError`).

2.4.2 Шаг 1. Среднее и центрирование

```
mu = X.mean(axis=0)
const_mask = np.all(X == X[0, :], axis=0)
if np.any(const_mask):
    mu = mu.copy()
    mu[const_mask] = X[0, const_mask]
Xc = X - mu
Xc[:, const_mask] = 0.0
```

- `mu` — вектор $\mu = \frac{1}{n} X^T \vec{1}$, обычное среднее по столбцам.
- `const_mask` — булева маска столбцов, где **все** значения точно совпадают (постоянные признаки). Сравнение точное (`==`), как требует методичка: «постоянство означает точное совпадение всех значений столбца».
- Для таких столбцов среднее переопределяется точным значением константы `c`, а центрированный столбец принудительно зануляется. Это защита от того, что `X - X.mean()` для константного столбца из-за арифметики с плавающей точкой может дать не строго нулевой, а исчезающе малый ненулевой разброс — который затем «испортил» бы и `scale` (получилась бы не единица, а крошечное число), и матрицу ковариаций.

2.4.3 Шаг 2. Масштаб (центрирование vs стандартизация)

```
if standardize:
    std = np.sqrt(np.sum(Xc ** 2, axis=0) / (n - 1))
    scale = np.where(const_mask, 1.0, std)
else:
    scale = np.ones(m)

Xs = Xc / scale
```

Прямая реализация формулы

$$s_j = \sqrt{\frac{1}{n-1} \sum_i \tilde{x}_{ij}^2} \text{ (непостоянный столбец)}, \quad s_j = 1 \text{ (постоянный столбец)}$$

Без стандартизации scale — вектор единиц, и $X_s = \tilde{X}$ (случай $D = I$ из методички). `np.where(const_mask, 1.0, std)` гарантирует единичный масштаб для константных признаков — иначе было бы деление на ноль (их `std`, после принудительного зануления X_c , равен 0).

2.4.4 Шаг 3-4. Матрица ковариаций / приём Грама и спектр

Дальше идёт развилка по условию `Gram_trick and n < m`.

Ветка приёма Грама (когда признаков больше, чем наблюдений, $n < m$):

```
G = (Xs @ Xs.T) / (n - 1)
eigval, eigvec = np.linalg.eigh(G)
order = np.argsort(eigval)[::-1]
eigval, eigvec = eigval[order], eigvec[:, order]
```

Вместо огромной $\Sigma_s \in \mathbb{R}^{m \times m}$ считаем маленькую матрицу Грама $G = \frac{1}{n-1} X_s X_s^\top \in \mathbb{R}^{n \times n}$ — это лемма о приёме Грама: ненулевая часть спектров G и Σ_s совпадает с учётом кратностей. `eigh` возвращает собственные значения **по возрастанию** (об этом отдельно предупреждает методичка), поэтому явно сортируем `[::-1]` — по убыванию.

```
tau = TAU_REL * np.max(np.abs(eigval)) if eigval.size else 0.0
if np.any(eigval < -tau):
    raise ValueError("Negative eigenvalue beyond numerical tolerance encountered.")

pos = eigval > tau
lam_pos, w_pos = eigval[pos], eigvec[:, pos]
```

Соглашение о численном нуле: $\tau_\lambda = 10^{-12} \max_i |\lambda_i|$. Значения меньше $-\tau_\lambda$ означают некорректный результат (матрица G теоретически неотрицательно определена — отрицательное значение сверх порога сигнализирует об ошибке), поэтому кидаем `ValueError`. Значения по модулю $\leq \tau_\lambda$ — просто нули; в приёме Грама векторы

для них вообще **не восстанавливаются**, как требует методичка: «*доставить базис направлений нулевой дисперсии не нужно*».

```
U_raw = (Xs.T @ w_pos) / np.sqrt((n - 1) * lam_pos)[None, :]
```

Это в точности формула из леммы о приёме Грама:

$$u = \frac{X_s^\top w}{\sqrt{(n-1)\lambda}}$$

применённая сразу ко всем столбцам `w_pos` (деление на массив `[None, :]` работает широковежательно по столбцам).

```
U_full = _mgs_orthonormalize(U_raw) if U_raw.shape[1] else U_raw
U_full = _fix_signs(U_full)

rnum = U_full.shape[1]
lam = np.zeros(m)
lam[:rnum] = lam_pos
```

Ортонормируем полученные векторы (см. выше), фиксируем знак, `rnum` — число полученных ненулевых компонент (численный ранг). Спектр `lam` дополняется нулями до длины m , как явно требует методичка: «*дополните спектр нулями до длины m* ».

Ветка напрямую через Σ_s (иначе, включая случай `Gram_trick=False`):

```
Sigma_s = (Xs.T @ Xs) / (n - 1)
eigval, eigvec = np.linalg.eigh(Sigma_s)
order = np.argsort(eigval)[::-1]
eigval, eigvec = eigval[order], eigvec[:, order]

tau = TAU_REL * np.max(np.abs(eigval)) if eigval.size else 0.0
if np.any(eigval < -tau):
    raise ValueError("Negative eigenvalue beyond numerical tolerance encountered.")

lam = np.where(np.abs(eigval) <= tau, 0.0, eigval)
rnum = int(np.sum(lam > 0))
U_full = _fix_signs(eigvec)
```

Здесь `eigh` сразу возвращает ортонормированный базис (в отличие от приёма Грама), поэтому MGS не нужен — только обнуление малых значений по порогу и фиксация знака. `lam` — полный спектр длины m ; обрезка не нужна, малые значения просто заменяются нулями на месте.

2.4.5 Шаг 5. Проверка k и обрезка

```

if not (1 <= k <= rnum):
    raise ValueError(
        f"k={k} is invalid: 1 <= k <= r_num={rnum} is required "
        f"(numerical rank r_num={rnum})."
    )

U = U_full[:, :k]
return mu, scale, U, lam

```

Как требует методичка: «При $k > r_{\text{num}}$ функция *rca_fit* завершается с ошибкой». Возвращается только первые k столбцов как U , но **весь** спектр λ_m длины m — так задана сигнатура функции.

2.5 pca_transform(X, mu, scale, U)

```
Xs = (X - mu) / scale
return Xs @ U
```

Прямая реализация $P = X_s U_k$: центрируем на **обучающих** μ , делим на **обучающий** scale , проецируем на **обучающие** оси U . Функция не пересчитывает μ/scale заново — она принимает их как параметры, что позволяет применять её и к новым (например, тестовым) данным без утечки информации («train/test leakage»): среднее и масштаб теста никогда не участвуют в вычислениях.

2.6 pca_inverse(P, mu, scale, U)

```
return (P @ U.T) * scale + mu
```

Обратное преобразование $\hat{X} = P U_k^T D + \tilde{1} \mu^T$ в обозначениях методички. Сначала возвращаемся из пространства компонент в пространство (стандартизованных) признаков через U^T , затем «расстандартизируем» — умножаем на scale (обратная операция делению в `pca_transform`), и наконец сдвигаем обратно на μ — из центра в исходные координаты. Порядок операций здесь ровно обратный порядку в `pca_transform`, что и делает эту пару функций взаимно обратными.

2.7 explained_variance_ratio(lam, k)

```
total = lam.sum()
if total <= 0:
    raise ValueError("Total variance (sum(lam)) must be positive.")
return float(lam[:k].sum() / total)
```

Реализует

$$EVR_k = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^m \lambda_i} = \frac{\sum_{i=1}^k \lambda_i}{\text{tr } \Sigma}$$

Проверка `total <= 0` соответствует условию методички «определим при $\text{tr } \Sigma > 0$ »: если все наблюдения совпадают, суммарная дисперсия равна нулю и отношение не определено — вместо тихого деления на ноль (которое дало бы NaN или inf) кидается явная ошибка.

2.8 Общие замечания по ядру

1. **Почему $n - 1$, а не n .** Поправка Бесселя: после центрирования одна степень свободы «уходит» — сумма отклонений по каждому признаку равна нулю, поэтому одно из n отклонений линейно выражается через остальные $n - 1$.

2. **Почему `eigh`, а не `eig`.** И Σ_s , и G всегда симметричны и неотрицательно определены. `eigh` использует эту структуру и гарантированно даёт вещественный, точный и, как правило, более быстрый результат по сравнению с общим `eig`, который не предполагает симметрию.
3. **Единый порог `tau`.** Вычисляется один раз от максимального по модулю собственного значения **после** сортировки и применяется единообразно и при обнулении спектра, и при определении `gnim`, и при проверке на «недопустимо отрицательные» значения (что теоретически невозможно для неотрицательно определённой матрицы и потому служит индикатором численной ошибки).

3 Часть 2. morphometry.py — морфометрия клеточных ядер

Данные: `sklearn.datasets.load_breast_cancer`, $n = 569$, $m = 30$, признаки различаются по масштабу на $\sim 2 \cdot 10^5$ раз. Везде используется `Gram_trick=False`, поскольку $n > m$ — приём Грама здесь не даёт вычислительного выигрыша.

print_top_weights — вспомогательная печать: берёт вектор весов u , сортирует индексы по убыванию $|u_i|$ и печатает топ- n признаков с наибольшим по модулю вкладом вместе со знаком веса.

task_1_2 — считает PCA в двух режимах (`standardize=False/True`), для каждого выводит спектр, $EV R_1, EV R_2, EV R_3$, топ-веса u_1, u_2 и строит scree-plot (график λ_i от номера i , в лог-масштабе). Далее — «диагноз»: для нестандартизованного варианта берутся два признака с наибольшими по модулю весами в u_1 . Модуль $|u_1^T e_j|$ считается напрямую как скалярное произведение u_1 с ортом e_j — он **численно совпадает** с $|u_{1j}|$, потому что $e_j^T u_1$ по определению скалярного произведения равно просто j -й координате u_1 . Затем считается $\sum (u_{1j})^2$ по этим двум признакам — доля их вклада в квадрат нормы $\|u_1\|^2 = 1$ (поскольку u_1 единичный, знаменатель равен 1, деление формально не нужно, но оставлено для наглядности формулы). Содержательный вывод: без стандартизации первая компонента почти целиком определяется признаками с наибольшей *абсолютной* дисперсией (площадь, периметр — они измеряются в единицах на 2–3 порядка больше остальных), а не признаками, наиболее значимыми содержательно.

task_3_scatter — для обоих режимов строит PCA с $k = 2$, проецирует данные (`pca_transform`) и рисует диаграмму рассеяния (P_1, P_2) , раскрашенную по истинной метке класса (метка используется только для цвета точек, не для построения компонент).

task_4_evr_curve — строит полный PCA ($k = m$), вычисляет $EV R_k$ для всех $k = 1, \dots, m$ и через `np.searchsorted` находит минимальное k , при котором $EV R_k \geq 0.90$ и ≥ 0.95 (`searchsorted` возвращает индекс первой позиции, где можно вставить искомое значение, сохранив сортировку возрастающего массива `evr`; +1 переводит 0-индексацию в номер компоненты).

task_5_interpret — печатает u_1, u_2 стандартизованного варианта, отсортированные по убыванию $||$, для содержательного чтения: какие признаки входят с одним знаком, какие — с противоположным.

task_6_stability — проверка устойчивости подпространств:

- Масштабы `scale_full` фиксируются один раз по **всей** стандартизованной выборке (вызовом `pca_fit` с $k=1$, нужны только `mu_full/scale_full`, но функция требует хотя бы $k = 1$).
- 50 раз данные случайно делятся пополам (`rng.permutation`), для каждой половины **своё** среднее пересчитывается заново (`Xi.mean(axis=0)`), но масштаб берётся общий, зафиксированный ранее — так и требует задание. PCA для каждой половины строится вручную через `eigh( $\Sigma$ )`, а не через `pca_fit`, именно потому что `pca_fit` сам пересчитал бы и `scale`, что здесь запрещено условием.

- Для $k = 1, 2$ считается

$$d_k = \left\| U_k^{(a)} (U_k^{(a)})^\top - U_k^{(b)} (U_k^{(b)})^\top \right\|_F$$

— расстояние Фробениуса между проекторами на подпространства, натянутые на первые k компонент в каждой половине. Это сравнение **подпространств**, а не отдельных векторов, поэтому не чувствительно к неоднозначности знака.

- Отдельно считается $|(u_1^{(a)})^\top u_1^{(b)}|$ и аналогично для u_2 — модуль косинуса угла между соответствующими направлениями в двух половинах (модуль — чтобы не зависеть от знака отдельного вектора).
- В конце печатаются среднее и стандартное отклонение d_1, d_2 по 50 повторам и фиксированный RNG_SEED — для воспроизводимости, как требует задание.

4 Часть 3. `eigenfaces.py` — собственные лица

Данные: `sklearn.datasets.fetch_olivetti_faces`, $n = 400$ изображений 64×64 , $m = 4096 \gg n$. Везде используется `Gram_trick=True`, так как именно этот случай ($n \ll m$) приём Грама и призван решать.

`show_images` — вспомогательная функция отрисовки: раскладывает список плоских векторов-изображений (`reshape` до $\text{IMG}=(64,64)$) в сетку `matplotlib` с подписями и сохраняет в PNG.

`task_1_mean_face` — считает среднее лицо μ (просто `X.mean(axis=0)`), отображает его как изображение, а также несколько центрированных лиц $\tilde{x}_i = x_i - \mu$.

`task_2_eigenfaces` — строит PCA с $k = 16$ через приём Грама, отображает столбцы U (каждый — вектор длины 4096) как изображения 64×64 : это и есть «собственные лица» — направления наибольшей дисперсии в пространстве изображений.

`task_3_spectrum` — берёт полный спектр `lam` (длины $m = 4096$, из которых реально ненулевых — не больше $n - 1 = 399$, поскольку $\text{rank}(\tilde{X}) \leq n - 1$ после центрирования), отбирает значения выше порога `tau`, строит их график в логарифмическом масштабе по оси λ и отдельно печатает число обнулённых значений.

`task_4_reconstruction` — для одного лица (`face_idx=10`) при разных $k \in \{1, 5, 10, 25, 50, \dots, 399\}$ считает PCA заново (при каждом k отдельно, так как `pca_fit` строит компоненты только с ненулевой в пределах порога дисперсией и принимает k как параметр), проецирует и восстанавливает лицо (`pca_transform+pca_inverse`), выкладывает реконструкции в ряд рядом с оригиналом. Далее — для **всей** выборки одним вызовом `pca_fit` с максимальным k считается эмпирическая ошибка восстановления

$$e_k = \frac{\|X - \hat{X}\|_F}{\|\tilde{X}\|_F}$$

и сравнивается с теоретической формулой $\sqrt{1 - EV R_k}$ на одном графике — как и требует задание, обе величины вычисляются по одной и той же выборке.

`task_5_compression` — считает число чисел, нужных для наивного хранения ($n \cdot m$), против хранения через PCA (μ размера m , U_k размера $m \times k$, проекции P размера $n \times k$) для разных k , и печатает коэффициент сжатия `naive/stored` — показывая, при каком k выигрыш от сжатия исчезает (когда `stored` приближается к `naive`).

`task_6_recognition` — классификатор ближайшего соседа:

- Для каждого из 40 человек 8 снимков идут в обучение, 2 — в тест (`rng.permutation` внутри каждого класса, фиксированный `RNG_SEED`).
- PCA строится **только** по обучающей части (`Gram_trick=True`, так как $n_{\text{train}} = 320 < m = 4096$).
- И обучающая, и тестовая часть проецируются через `pca_transform` с параметрами, полученными на обучении (то есть вычитается **среднее обучающей** выборки — как явно требует задание).
- Для каждого тестового снимка ищется ближайший (по евклидову расстоянию в пространстве проекций) обучающий снимок, его метка присваивается тестовому (при равных расстояниях `np.argmin` берёт первый по индексу — соответствует требованию «выберите первый по индексу»).

- Точность считается для набора значений k и строится график accuracy(k).

task_7_distance_to_face_space — реализует

$$d(x) = \|\tilde{x} - U_k U_k^\top \tilde{x}\|_2, \quad \tilde{x} = x - \mu_{train}$$

— расстояние от изображения до аффинного подпространства лиц при фиксированном $k = 50$. Генерируются «посторонние» изображения (случайный шум и однотонные прямоугольники) и берётся выборка настоящих лиц; для обеих групп считается $d(x)$, каждая группа делится на часть для подбора порога и часть для проверки. Порог d_0 берётся как среднее двух медиан (простая эвристика), после чего на проверочной части считаются доля отвергнутых (ложно) лиц и доля посторонних изображений, ошибочно принятых за лица — и строится гистограмма распределений $d(x)$ для обеих групп с отмеченным порогом.

task_8_timing — сравнивает время построения компонент напрямую через Σ (4096×4096) и приёмом Грама (400×400) при одинаковом $k = 50$, используя `time.perf_counter()`. В обоих случаях в измеряемое время включены построение соответствующей матрицы и `eigh`, но не загрузка данных. После этого сравниваются полученные собственные значения ($\max |\Delta\lambda|$) и направления с учётом знака — фиксация знака (`_fix_sign_local`, локальная копия той же логики) нужна, потому что иначе противоположно направленные, но математически эквивалентные векторы дали бы ложно большое расхождение при прямом сравнении координат.

5 Часть 4. manifold.py — рукописные цифры и многообразия

Данные: `sklearn.datasets.load_digits`, $n = 1797$, изображения 8×8 ($m = 64$), 10 классов. Везде `Gram_trick=False`, так как $n > m$.

task_1_scatter_and_distances — строит PCA с $k = 2$, проецирует все данные, рисует диаграмму рассеяния ($P_{\cdot 1}, P_{\cdot 2}$) с раскраской по классам (`c=y`, `map="tab10"`). Затем для каждого класса считается центроид (среднее проекций точек этого класса) в 2D-пространстве, и через широковежательное вычитание `centroids[:, None, :] - centroids[None, :, :]` с последующей нормой по последней оси строится полная 10×10 матрица попарных евклидовых расстояний между центроидами классов — она печатается в виде таблицы.

task_2_dimension — аналогично `task_4_evr_curve` из морфометрии: строит полный PCA ($k = m = 64$), находит минимальное k для $EVR_k \geq 0.90$ и ≥ 0.95 через `np.searchsorted`, печатает и сравнивает с исходной размерностью 64. Отдельно поясняется (как и требует методичка), что число компонент, нужных для сохранения доли **дисперсии**, — это не то же самое, что внутренняя (возможно, нелинейная) размерность многообразия, на котором лежат данные: PCA — линейный метод и видит только линейные корреляции между признаками.

task_3_3d_projection — строит PCA с $k = 3$, проецирует данные и рисует трёхмерную диаграмму рассеяния (`Axes3D`, `scatter` с `c=y`). В комментарии явно разделяются два разных утверждения: визуальное наблюдение по картинке («в 3D некоторые классы визуально лучше разделены, чем в 2D») и строгий вывод о качестве классификации — второе картинкой не доказывается, задание прямо просит не путать эти вещи.

6 Сквозные замечания по всем файлам

- **RNG_SEED** используется единообразно во всех местах со случайностью (разбиение выборки в `morphometry.py::task_6_stability` и в `eigenfaces.py::task_6_recognition`, генерация «посторонних» изображений в `task_7_distance_to_face_space`) — это прямое требование методички: «укажите начальное состояние генератора случайных чисел для воспроизводимости».
- **Gram_trick** включается или выключается в зависимости от соотношения n и m в конкретном наборе данных: `False` там, где $n > m$ (морфометрия, цифры), `True` там, где $n \ll m$ (лица) — это соответствует рекомендации методички использовать приём Грама именно в случае $n < m$, где прямое построение $\Sigma \in \mathbb{R}^{m \times m}$ дорого или невозможно по памяти.
- **Разделение обучения и применения:** везде, где есть train/test-разбиение (`eigenfaces.py::task_6_recognition`, `task_7`), среднее, масштаб и компоненты вычисляются **только** по обучающей части, а `pca_transform` затем применяется к обеим частям без пересчёта этих параметров — так исключается утечка информации из теста в обучение.